

Algorithms

Divide and Conquer

Dr. Mudassir Shabbir

LUMS University

March 11, 2026



Dynamic Programming



What is Dynamic Programming?

Definition

Dynamic Programming (DP) is an algorithmic paradigm for solving **optimization problems** by:

- 1 Breaking the problem into overlapping subproblems.
- 2 Solving each subproblem *once* and storing the result.
- 3 Using stored results to avoid redundant recomputation.



What is Dynamic Programming?

Definition

Dynamic Programming (DP) is an algorithmic paradigm for solving **optimization problems** by:

- 1 Breaking the problem into overlapping subproblems.
- 2 Solving each subproblem *once* and storing the result.
- 3 Using stored results to avoid redundant recomputation.

Key phrase: *"Storing solutions to subproblems."*



What is Dynamic Programming?

Definition

Dynamic Programming (DP) is an algorithmic paradigm for solving **optimization problems** by:

- 1 Breaking the problem into overlapping subproblems.
- 2 Solving each subproblem *once* and storing the result.
- 3 Using stored results to avoid redundant recomputation.

Key phrase: *"Storing solutions to subproblems."*

Relation to other paradigms

- **Divide & Conquer:** subproblems are *independent*; combine their solutions.
- **Greedy:** make local optimal choice; no subproblem storage.

Two Key Conditions for DP

1. Optimal Substructure

- An optimal solution to the problem contains optimal solutions to subproblems.
- The global optimum can be computed from optimal subproblem solutions.



Two Key Conditions for DP

1. Optimal Substructure

- An optimal solution to the problem contains optimal solutions to subproblems.
- The global optimum can be computed from optimal subproblem solutions.

2. Overlapping Subproblems

- The same subproblem arises many times (not just once).



Two Key Conditions for DP

1. Optimal Substructure

- An optimal solution to the problem contains optimal solutions to subproblems.
- The global optimum can be computed from optimal subproblem solutions.

2. Overlapping Subproblems

- The same subproblem arises many times (not just once).

If subproblems don't overlap:

Use Divide & Conquer instead (DP caching provides no benefit).



Example: Fibonacci

Naive Recursive Version

```
function Fib( $n$ )  
  if  $n \leq 1$  then return  $n$   
  return Fib( $n - 1$ ) + Fib( $n - 2$ )
```



Example: Fibonacci

Naive Recursive Version

```
function Fib(n)
  if  $n \leq 1$  then return  $n$ 
  return Fib( $n - 1$ ) + Fib( $n - 2$ )
```

Problem: Fib($n - 1$) calls Fib($n - 2$), and so does the Fib($n - 2$) term.



Example: Fibonacci

Naive Recursive Version

```
function Fib(n)
  if  $n \leq 1$  then return  $n$ 
  return Fib( $n - 1$ ) + Fib( $n - 2$ )
```

Problem: Fib($n - 1$) calls Fib($n - 2$), and so does the Fib($n - 2$) term.

Exponential time: $T(n) = O(2^n)$.



Example: Fibonacci

Naive Recursive Version

```
function Fib(n)
  if  $n \leq 1$  then return  $n$ 
  return Fib( $n - 1$ ) + Fib( $n - 2$ )
```

Problem: Fib($n - 1$) calls Fib($n - 2$), and so does the Fib($n - 2$) term.

Exponential time: $T(n) = O(2^n)$.

DP Fix: Memoization

```
array memo
function Fib-Memo(n)
  if memo[n] is set then return memo[n]
  if  $n \leq 1$  then result  $\leftarrow n$ 
  else result  $\leftarrow$  Fib-Memo( $n - 1$ ) + Fib-Memo( $n - 2$ )
  memo[n]  $\leftarrow$  result; return result
```



Example: Fibonacci

Naive Recursive Version

```
function Fib(n)
  if  $n \leq 1$  then return  $n$ 
  return Fib( $n - 1$ ) + Fib( $n - 2$ )
```

Problem: Fib($n - 1$) calls Fib($n - 2$), and so does the Fib($n - 2$) term.

Exponential time: $T(n) = O(2^n)$.

DP Fix: Memoization

```
array memo
function Fib-Memo(n)
  if memo[n] is set then return memo[n]
  if  $n \leq 1$  then result  $\leftarrow n$ 
  else result  $\leftarrow$  Fib-Memo( $n - 1$ ) + Fib-Memo( $n - 2$ )
  memo[n]  $\leftarrow$  result; return result
```

Each subproblem solved once: $O(n)$ time and space.



Bottom-Up vs Top-Down

Approach 1: Top-Down (Memoization)

- Start with the original problem.
- Recursively solve subproblems, storing results.
- Looks like a tree pruned by memo checks.



Bottom-Up vs Top-Down

Approach 1: Top-Down (Memoization)

- Start with the original problem.
- Recursively solve subproblems, storing results.
- Looks like a tree pruned by memo checks.

Approach 2: Bottom-Up (Tabulation)

- Solve subproblems in an order where dependencies come first.
- Use a table to store results iteratively.
- No recursion stack overhead.



Bottom-Up vs Top-Down

Approach 1: Top-Down (Memoization)

- Start with the original problem.
- Recursively solve subproblems, storing results.
- Looks like a tree pruned by memo checks.

Approach 2: Bottom-Up (Tabulation)

- Solve subproblems in an order where dependencies come first.
- Use a table to store results iteratively.
- No recursion stack overhead.

Fibonacci (Bottom-Up):

```
function Fib-Tab( $n$ )  
    DP[0]  $\leftarrow$  0; DP[1]  $\leftarrow$  1  
    for  $i = 2$  to  $n$  do  
        DP[ $i$ ]  $\leftarrow$  DP[ $i - 1$ ] + DP[ $i - 2$ ]  
    return DP[ $n$ ]
```



Example: Coin Change Problem

Problem: Given coins of denominations $[c_1, c_2, \dots, c_k]$, find the **minimum number of coins** to make amount n .



Example: Coin Change Problem

Problem: Given coins of denominations $[c_1, c_2, \dots, c_k]$, find the **minimum number of coins** to make amount n .

Optimal Substructure:

$$\text{minCoins}(n) = 1 + \min_{i: c_i \leq n} \{\text{minCoins}(n - c_i)\}.$$

If the best way to make n uses coin c_i , then the rest must optimally make $n - c_i$.



Example: Coin Change Problem

Problem: Given coins of denominations $[c_1, c_2, \dots, c_k]$, find the **minimum number of coins** to make amount n .

Optimal Substructure:

$$\text{minCoins}(n) = 1 + \min_{i: c_i \leq n} \{\text{minCoins}(n - c_i)\}.$$

If the best way to make n uses coin c_i , then the rest must optimally make $n - c_i$.

DP Table (example: coins = [1,2,5], target $n = 11$):

Amount	0	1	2	3	4	5	6	7	8	9	10	11
DP	0	1	1	2	2	1	2	2	3	3	2	3



Example: Coin Change Problem

Problem: Given coins of denominations $[c_1, c_2, \dots, c_k]$, find the **minimum number of coins** to make amount n .

Optimal Substructure:

$$\text{minCoins}(n) = 1 + \min_{i: c_i \leq n} \{\text{minCoins}(n - c_i)\}.$$

If the best way to make n uses coin c_i , then the rest must optimally make $n - c_i$.

DP Table (example: coins = [1,2,5], target $n = 11$):

Amount	0	1	2	3	4	5	6	7	8	9	10	11
DP	0	1	1	2	2	1	2	2	3	3	2	3

Answer: **3 coins** (e.g., 5+5+1).



Coin Change Pseudocode

```
procedure CoinChange(coins,  $n$ )  
   $DP[0] \leftarrow 0$   
  for  $i = 1$  to  $n$  do  
     $DP[i] \leftarrow \infty$   
    for each coin  $c$  in coins do  
      if  $c \leq i$  then  
         $DP[i] \leftarrow \min(DP[i], DP[i - c] + 1)$   
  return  $DP[n]$ 
```



Coin Change Pseudocode

```
procedure CoinChange(coins,  $n$ )  
   $DP[0] \leftarrow 0$   
  for  $i = 1$  to  $n$  do  
     $DP[i] \leftarrow \infty$   
    for each coin  $c$  in coins do  
      if  $c \leq i$  then  
         $DP[i] \leftarrow \min(DP[i], DP[i - c] + 1)$   
  return  $DP[n]$ 
```

Time: $O(n \times k)$ where $k = |\text{coins}|$.

Space: $O(n)$ for the DP table.



Example: Longest Common Subsequence (LCS)

Problem: Given two strings X and Y , find the longest subsequence that appears in both.



Example: Longest Common Subsequence (LCS)

Problem: Given two strings X and Y , find the longest subsequence that appears in both.

Example: $X = \text{AGGTAB}$, $Y = \text{GXTXAYB}$. $\text{LCS} = \text{GTAB}$ (length 4).



Example: Longest Common Subsequence (LCS)

Problem: Given two strings X and Y , find the longest subsequence that appears in both.

Example: $X = \text{AGGTAB}$, $Y = \text{GXTXAYB}$. $\text{LCS} = \text{GTAB}$ (length 4).

Optimal Substructure:

$$\text{LCS}(i, j) = \begin{cases} 1 + \text{LCS}(i-1, j-1) & \text{if } X[i] = Y[j] \\ \max(\text{LCS}(i-1, j), \text{LCS}(i, j-1)) & \text{otherwise} \end{cases}$$



Example: Longest Common Subsequence (LCS)

Problem: Given two strings X and Y , find the longest subsequence that appears in both.

Example: $X = \text{AGGTAB}$, $Y = \text{GXTXAYB}$. $\text{LCS} = \text{GTAB}$ (length 4).

Optimal Substructure:

$$\text{LCS}(i, j) = \begin{cases} 1 + \text{LCS}(i - 1, j - 1) & \text{if } X[i] = Y[j] \\ \max(\text{LCS}(i - 1, j), \text{LCS}(i, j - 1)) & \text{otherwise} \end{cases}$$

DP table is $(m + 1) \times (n + 1)$ where $m = |X|$, $n = |Y|$.

Time: $O(m \times n)$.



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS =?

	ϵ	B	D	C	A	B
ϵ	?					
A						
B						
C						
B						



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS =?

	ϵ	B	D	C	A	B
ϵ	0	?	?	?	?	?
A						
B						
C						
B						



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS =?

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	?					
B	?					
C	?					
B	?					



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS =?

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	0	?				
B	0					
C	0					
B	0					



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS =?

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	0	0	?			
B	0					
C	0					
B	0					



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS =?

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	0	0	0	?		
B	0					
C	0					
B	0					



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS =?

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	0	0	0	0	?	
B	0					
C	0					
B	0					



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS =?

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	0	0	0	0	1	?
B	0					
C	0					
B	0					



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS =?

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	?				
C	0					
B	0					



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS =?

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	1	?			
C	0					
B	0					



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS =?

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	1	1	?		
C	0					
B	0					



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS =?

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	1	1	1	?	
C	0					
B	0					



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS =?

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	1	1	1	1	?
C	0					
B	0					



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS =?

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	1	1	1	1	2
C	0	?				
B	0					



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). $\text{LCS} = ?$

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	1	1	1	1	2
C	0	1	?			
B	0					



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS =?

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	1	1	1	1	2
C	0	1	1	?		
B	0					



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS =?

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	1	1	1	1	2
C	0	1	1	2	?	
B	0					



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS =?

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	1	1	1	1	2
C	0	1	1	2	2	?
B	0					



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS =?

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	1	1	1	1	2
C	0	1	1	2	2	2
B	0	?				



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS =?

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	1	1	1	1	2
C	0	1	1	2	2	2
B	0	1	?			



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS =?

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	1	1	1	1	2
C	0	1	1	2	2	2
B	0	1	1	?		



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS =?

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	1	1	1	1	2
C	0	1	1	2	2	2
B	0	1	1	2	?	



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS =?

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	1	1	1	1	2
C	0	1	1	2	2	2
B	0	1	1	2	2	?



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS = ?

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	1	1	1	1	2
C	0	1	1	2	2	2
B	0	1	1	2	2	3



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). LCS = ?

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	1	1	1	1	2
C	0	1	1	2	2	2
B	0	1	1	2	2	3

LCS length = $\text{LCS}(4, 5) = \mathbf{3}$. One solution: BCB (matched at $X[2], X[3], X[4]$ with $Y[1], Y[3], Y[5]$).



LCS: DP Table

Example: $X = \text{ABCB}$ (length 4), $Y = \text{BDCAB}$ (length 5). $\text{LCS} = ?$

	ϵ	B	D	C	A	B
ϵ	0	0	0	0	0	0
A	0	0	0	0	1	1
B	0	1	1	1	1	2
C	0	1	1	2	2	2
B	0	1	1	2	2	3

$\text{LCS length} = \text{LCS}(4, 5) = \mathbf{3}$. One solution: BCB (matched at $X[2], X[3], X[4]$ with $Y[1], Y[3], Y[5]$).

Match vs. no-match

When $X[i] = Y[j]$: diagonal +1. Otherwise: max of left and top neighbor.